

* **JDBC** - Java Data Base Connectivity.

Ex:

* package com.dml;

```
public class DMLOperations {
```

```
    public static void main (String [ ] args) {
```

(i) Register your driver (optional).

(ii) Establish a connection.

[url, (Username, password)]

↓
* It helps us to connect with database

[Jdbc:mysql://localhost:3306 - sql workbench url.]

(iii) Create a statement.

(iv) Prepare your SQL Query (Case insensitive)

(v) Execute the Query.

(vi) Close the statement & connection (Resources).

* **i) driver**

→ (Connecting you from one connection to other)

→ Here, this is a translator b/w Java & SQL.

→ It is coming from MySQL.

~~MySQL~~ MySQL connector, Oracle connector, MongoDB connector.

ii) Connection

→ Link (kind of internet/wifi)

→ live connection b/w Java & SQL

iii) Statement

→ Java object

→ DDL, DML, DQL - used to do all actual operations.

* **Build Tools** - Used to manage Dependency.

Ex: Maven, Gradle, Bazel.

```
package com.dml; import java.sql.java.*; DriverManager;
public class DmlOperations {
    public static void main(String[] args) throws ClassNotFoundException
    & Exception, SQL Exception, {
```

(i) // Register the driver.

(connector.java)

checked
Exception
found ⇒ Class.forName ("com.mysql.cj.jdbc.Driver"); output:
package class Exception
SYSO ("Driver Registered");

// maven repository (google it) → mysql connector → mysql-connector-j

⇒ jar - Java archive file

→ Executable file.

To add Jar file

⇒ Rightclick on project

— Build path

→ Configure build path

→ Libraries

→ Module path

→ Add External Jars

→ Add that downloaded Jar.

→ Apply / Apply & close.

(ii) // Establish connection.

String url = "jdbc:mysql://localhost:3306/film-adv-java";

String username = "root";

↓
database
Name

String password = "GayatriRavikumar@";

Connection

Connection = DriverManager.getConnection(url, username, password);

(It returns Connection)

(iii) // Create a statement.

Statement = Connection.createStatement();

(iv) // Prepare SQL Query

film-adv-java.employees — SQL Query.

String query = "insert into employees values (4, 'abc@gmail.com', 30000)";

v) Execute the Query.

statement.executeUpdate(query);

Output:

Driver Registered
Inserted Data.

SYSO ("Inserted Data");

String query2 = "Update employees set salary = 20000 where empid = 6";

statement.executeUpdate(query2);

String query3 = "Delete from employees where empid = 6";

statement.executeUpdate(query3);

Class-57

15/07/2025

DQL Operations:

package com.dql;

public class DQLOperations {

public static void main(String[] args) {
try {

JdbcUtil jdbcUtil = new JdbcUtil();

Connection

connection = jdbcUtil.getConnection();

if (connection == null)

catch (Exception e) {

System.out.println(e.getMessage());

}

Statement statement = connection.createStatement();

String query = "select * from employees";

// when we do select operation in java, it will return Result set (look like a table)

↓ = statement.^{Query}~~executeUpdate~~(query) (∴ DQL Operations)

ResultSet rs = statement.executeQuery("employeeId | email | salary"); (1 tab space)

// to take out data, we use,

while (rs.next()) {

~~rs.get~~ int empid = rs.getInt(1);

String email = rs.getString(2);

double salary = rs.get^{Double}~~String~~("salary");

System.out.println("Employee Id | email | salary");

System.out.println(empid + email + salary);

}
catch (Exception e) {

System.out.println(e.getMessage());

}
}

```
package com.util;
```

```
public class JdbcUtil {  
    public Connection getConnection() throws SQLException = null;  
    public Connection getConnection() throws SQLException {
```

```
        // String url = "jdbc:mysql://localhost:3306/plm";
```

```
        // String username = "root";
```

```
        // String password = "Gayatriaravikumar@123";
```

```
        if (Connection == null) {
```

```
            Connection connection = DriverManager.getConnection(  
                Constants.url, Constants.Username, Constants.password);
```

```
            return connection;
```

```
        }
```

```
package com.util;
```

```
public class Constants {
```

```
    public static final String url = "jdbc:mysql://localhost:3306/plm";
```

```
    public static final String Username = "root";
```

```
    public static final String password = "Gayatriaravikumar@123";
```

```
}
```

* To create tables in Java.

```
package com.dell;
```

```
public class UsersTable {
```

```
    public static void main(String [] args) {
```

```
        JdbcUtil jdbcUtil = new JdbcUtil();
```

```
        Connection connection = jdbcUtil.getConnection();
```

```
        Statement statement = connection.createStatement();
```

```
        String query = "create table users (username varchar(20),  
                        password varchar(30))";
```

```
        statement.executeUpdate(query);
```

```
        System.out.println("Table created");
```

```
    }
```

```
}
```

```

package com.ddl;

public class Signup {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Syso("Enter UserName:");
        String username = sc.nextLine();
        Syso("Enter Password");
        String password = sc.nextLine();
        JdbcUtil jdbcUtil = new JdbcUtil();
        Connection connection = jdbcUtil.getConnection();
        Statement statement = connection.createStatement();

        String Query = "insert into users values ('+username+', "+
            "'password'+")";
        statement.executeUpdate(query);
        Syso("inserted");
    }
}

```

```

package com.ddl;

public class Login {
    public static void main {

```

Same as above upto statement.

```

String query = "Select * from users where username = ' " +
    username + " ' and password = ' " + password + " '";

```

```

ResultSet rs = statement.executeQuery(query);
if (rs.next()) {
    Syso("Logged In");
} else {
    Syso("Invalid Credentials");
}

```

SQL Injection:

- to avoid this SQL Injection, introduced Prepared Statement.
(we do not concatenate anything).
- Prepared statement depend on SQL Query.

```
public static void preparedStatement ( ) {
```

Same as above till connection,

```
String query = "Select  
Connection.prepareStatement ( )
```

```
String query = "Select * from users where username = ? And  
password = ?";
```

Prepared Statement

```
statement = connection.prepareStatement (query);
```

```
statement.setString (1, username);
```

```
statement.setString (2, password);
```

ResultSet

```
rs = statement.executeQuery ( );
```

```
}
```

```
psvm {
```

```
preparedStatement ( );
```

class-58

```
}
```

// Metadata: Data about data.

→ It comes from Result set (get it).

Ex: package com.metadata;

```
public class MetaDataInfo {
```

```
public static void main (String [] args) {
```

```
JdbcUtil jdbcUtil = new JdbcUtil ( );
```

Connection
connection = jdbcUtil.getConnection ();

Prepare
Statement statement = connection.^{prepare}executeStatement (query);

```
String query = "select * from employees";
```

```
ResultSet rs = statement.executeQuery ( );
```

16/07/2025

```
while (rs.next()) {
```

```
    Sysout(rs.getInt(1) + "\t" + rs.getString(2) + "\t" +  
        rs.getDouble(3));
```

```
}
```

(8)

```
rs.get(returns result set)
```

```
ResultSetMetaData metaData = result rs.getMetaData();
```

```
// we can see column count,
```

```
* Sysout("Column count: " + metaData.getColumnCount());
```

```
// to see column Names.
```

```
* Sysout("ColumnName: " + metaData.getColumnName(1) + "\t" +  
    metaData.getColumnCountCount(2) + "\t" +  
    metaData.getColumnCountCount(3));
```

(8)

```
int count = metaData.getColumnCount();
```

```
for (int i=1, i<=count, i++) {
```

```
    Sysout(metaData.getColumnName(i));
```

```
    Sysout("\t");
```

```
}
```

```
Sysout(metaData.getTableName());
```

```
Sysout(metaData.getColumnType(1));
```

```
Sysout(metaData.getColumnTypeName(1));
```

```
Sysout(metaData.getColumnTypeName(2));
```

```
}
```

ex:

```
package com.tcl;
```

```
public class TCLOperations {
```

```
    public static void main(String[] args) {
```

```
        try { Connection connection = null;
```

```
            JdbcUtil jdbcUtil = new JdbcUtil();
```

```
            Connection connection = jdbcUtil.getConnection();
```

```
            // to turn off autocommit;
```

```
            connection.setAutoCommit(false);
```

```
            PreparedStatement statement = connection.prepareStatement(
```

```
                "update employees set salary = ? where empId = ?");
```

```
            statement.setDouble(1, 10000);
```

```
            statement.setInt(2, 1);
```

```
            statement.executeUpdate();
```

```
            Syso("Updated...");
```

```
            connection.commit();
```

```
            connection.rollback();
```

```
        }
```

```
        catch (Exception e) {
```

```
            connection.rollback();
```

```
        }
```

```
    }
```

```
}
```

(∴ we cannot
rollback after
commit)

Ex:

```
package com.batch;
```

```
public class BatchProcessing {
```

```
    psvm {
```

```
        JdbcUtil jdbcUtil = new JdbcUtil();
```

```
        Connection connection = jdbcUtil.getConnection();
```

```
        Connect
```

```
        Prepared Statement statement =  
            connection.prepareStatement("Insert into employees values  
            (?, ?, ?)");
```

```
        statement.setInt(1, 1);
```

```
        statement.setString(2, "Swathi@gmail.com");
```

```
        statement.setDouble(3, 10000);
```

***** // to insert multiple rows, use Batches.

```
        statement.addBatch();
```

```
        statement.setInt(1, 8);
```

```
        statement.setString(2, "Gayatri@gmail.com");
```

```
        statement.setDouble(3, 50000);
```

```
        statement.addBatch();
```

***** statement.executeUpdate(); [It returns int array[]]

```
        sysout("Added Batch");
```

```
    }
```

```
}
```

Ex: package com.callable;
public class StoredProcedures {

psvm {

JdbcUtil jdbcUtil = new JdbcUtil();

Connection connection = jdbcUtil.getConnection();

CallableStatement statement = connection.prepareStatement("{call getAllEmployees()}");

ResultSet rs = statement.executeQuery();

while(rs.next()) {
 syso(rs.getInt(1) + "\t" + rs.getString(2) + "\t" +
 rs.getDouble(3));

} // In Parameters

CallableStatement statement2 =
= connection.prepareStatement("{call getEmployeeById(?, ?)}");

statement2.setInt(1, 1);

statement2.setString(2, "fayaz@gmail.com");

ResultSet rs = statement.executeQuery();

// Out parameters

CallableStatement

statement3 = connection.prepareStatement("{call getSalaryById(?, ?)}");

statement3.setInt(1, 1);

statement3.registerOutParameter(2, Types.Decimal);

ResultSet rs = statement3.execute();

double salary = statement3.getDouble(2);

syso("salary: " + salary);

In Out Parameters:

CallableStatement
statement4 = connection.prepareStatement("{call increaseSalary
(?) ?}");

statement4.setDouble(1, 1000);

statement4.registerOutParameter(1, Types.Decimal);

statement4.execute();

double salary = statement4.getDouble(1); syso(salary);